# UniCLOB

## A Unified AMM–Order Book

Persistent liquidity and native price–time execution

Technical paper · 20 October 2026

**Abstract**

UniCLOB is a fully on-chain exchange that integrates a concentrated-liquidity AMM and a central limit order book into one price-ordered execution engine. Continuous liquidity and discrete orders form a common supply curve: the AMM executes to the next resting price, the FIFO queue supplies additional quantity at that price, and the curve resumes when the queue is exhausted. Passive providers retain persistent, reversible range positions; active makers retain explicit price, quantity, cancellation, and queue priority. A single taker fee applies throughout, with source-dependent allocation to liquidity providers and the protocol.

The design compresses the two forms of liquidity independently while matching them jointly. Range positions require two endpoints regardless of width. Packed quantity trees allow complete order levels to execute without visiting individual makers. Hierarchical price indexes skip empty price space, and singleton custody permits net settlement across operations. The resulting cost model depends on visited liquidity boundaries and price levels, rather than the width of an LP range or the number of makers in a fully consumed level.

We specify the matching and accounting rules, describe the EVM implementation and integration contracts, and compare LP and taker workflows with published Uniswap artifacts, alongside separate measurements of native maker workflows. Sparse traversal and aggregate fills keep execution costs low across large price gaps and large same-price queues. Dense boundary crossings, new position creation, and repeated quote replacement remain material costs.

## 1. Introduction

Automated liquidity and a central limit order book solve different parts of the exchange problem. A concentrated-liquidity position commits capital to a reusable trading rule over a price interval. A limit order commits a finite quantity to one side of the market at a chosen price. The former supplies persistent inventory without continuous quote maintenance; the latter gives a maker direct control over execution price, size, and cancellation. A market supporting both should preserve these commitments while exposing their combined liquidity to a single trade.

UniCLOB implements that market as a unified execution engine. Its continuous AMM is the underlying price path. Ordinary orders add discrete executable quantity along that path. An LP supplies a range and leaves received inventory available for reverse execution. An active maker supplies funded bids or asks and claims proceeds when they fill. A taker encounters both under one price-ordering rule, input budget, fee accumulator, and settlement operation.

The principal difficulty is representation. Expanding every range into individual orders makes position management proportional to range width. Updating every maker during a sweep makes taker execution proportional to the number of filled orders. UniCLOB avoids both expansions. Analytical range inventory and endpoint accounting compress the AMM; aggregate FIFO quantity trees compress the book. Matching joins these representations at their next common execution event.

The design has four principal properties:

1. **Unified execution.** Curve liquidity and price-time queues are matched within the same state transition. No external routing decision is required between the two sources inside a market.
2. **Persistent passive liquidity.** Every aligned interval, including full range, uses two endpoints. Inventory remains committed to the LP's rule until liquidity is removed; fills do not turn a range into exhausted ordinary orders.
3. **Aggregate order execution.** A fully consumed price level updates an aggregate root, while individual proceeds remain claimable. Taker work does not grow linearly with the maker count at that level.
4. **Common fees and settlement.** Takers pay one cumulative input fee across sources. Source-dependent fee allocation, reserve accounting, and transient wallet netting preserve the distinct claims of LPs and makers.

Three choices govern priority. Executable price comes first: the matcher cannot pass a resting order to obtain a worse price from the curve. Curve liquidity between the current price and an ordinary order must execute before that order. At the resting price, ordinary liquidity executes in FIFO order. This is the precise meaning of **AMM priority** in a continuous market; it does not assign a finite AMM quantity to every tick.

Efficiency here has a specific meaning. Position management is independent of range width, empty price space is skipped, and complete-level fills do not visit each maker. The implementation still pays for distinct initialized boundaries, ordinary prices, storage updates, and token settlement. Section 11 measures those costs rather than assuming that a hybrid must be cheaper on every operation.

The equations describe the implemented mechanism. Measurements use implementation revision 27e03b0. The contracts have undergone local testing and internal review; independent audit and public-chain operational evaluation remain outstanding.

## 2. Relationship to existing designs

Uniswap v3 represents liquidity by intervals, changes active liquidity at initialized endpoints, and accounts for fees through growth accumulators. Its ticks are boundaries in a curve representation, rather than queues of independently cancellable orders. UniCLOB adopts that position model and adds ordinary price-time queues to the execution path [1].

Uniswap v4 combines concentrated liquidity with singleton custody, deferred settlement, and programmable hooks. UniCLOB fixes the joint AMM–FIFO transition in the core and uses a bounded wallet-settlement interface. Related combinations can be expressed through v4 hooks; the contribution here is the specified matching, ownership, and cost model, rather than the mere coexistence of ranges and orders. There is no hook interface [2].

Aggregate order execution with deferred individual settlement has precedent in on-chain order books. Clober's LOBSTER design uses segment trees to support on-chain matching. UniCLOB uses a packed four-way quantity tree and combines it with continuous AMM execution. Tree-based order aggregation itself is not a novelty claim [3].

An external router could split a trade between an AMM and a separate order book. Here, the source boundary is part of the market's state transition: both sources share a price path, budget, fee accumulator, and atomic reserve update. This removes a routing decision between those two sources within a market. Routing across markets and venues remains an integration task.

## 3. Market state and units

A market exchanges token0, denoted $X$, and token1, denoted $Y$. All contract amounts are integer token atoms. Price is quote atoms per base atom:

$$\begin{aligned} P &= Y/X, \\ s &= \sqrt{P}, \\ P(t) &= 1.0001^t. \end{aligned} \tag{1}$$

The stored square-root price is Q64.96. For tokens with decimal counts $d_X, d_Y$, the displayed price in whole tokens is $P\,10^{d_X-d_Y}$. Decimal normalization belongs in the integration layer; the matcher operates on atoms.

Each market fixes the ordered token addresses, tick spacing $h$, base lot size $q$, taker fee $f$ in parts per million, protocol divisor $d$, and initial square-root price. Its identifier is `keccak256(abi.encode(config))`. Initial price is part of that identity. Thus a token pair and fee do not uniquely identify a market, and the contract does not sort token addresses for the caller.

| Symbol | Meaning |
|---|---|
| $s$ | Current square-root price |
| $a, b$ | Lower and upper square-root endpoints of a position |
| $L$ | Active aggregate liquidity, or one position's liquidity when specified |
| $h$ | Tick spacing for LP endpoints and ordinary orders |
| $q$ | Token0 atoms per ordinary-order lot |
| $M$ | Fee denominator, $10^6$ |
| $f$ | Taker fee numerator, in ppm |
| $d$ | Protocol share denominator on AMM fees |
| $N$ | Cumulative principal input consumed by a trade |
| $G(N)$ | Gross input required to fund $N$ |
| $H_X, H_Y$ | Global Q128 LP fee-growth accumulators |

The market stores the current square-root price, directional current tick, active liquidity, fee growth, and separate AMM and order reserves. Three sparse indexes locate initialized LP endpoints, asks, and bids. Positions and orders are owner-addressed records. Free balances and protocol fees are accounted for at the singleton level by token.

Market creation is permissionless. Valid spacing is 1 through 32,767; fees are positive and at most 100,000 ppm; the protocol divisor is between 2 and 10, subject to the reference schedule in Section 6. Initialization requires a price strictly inside the supported square-root domain. Configuration cannot subsequently be changed.

## 4. Persistent concentrated liquidity

### 4.1 Inventory

For one position with liquidity $L$ and square-root interval $[a, b)$, principal inventory in exact arithmetic is

$$(x(s), y(s)) = \begin{cases} (L(1/a - 1/b), 0), & s \le a, \\ (L(1/s - 1/b), L(s-a)), & a < s < b, \\ (0, L(b-a)), & s \ge b. \end{cases} \tag{2}$$

While active, the translated reserves satisfy

$$(x + L/b)(y + La) = L^2. \tag{3}$$

A trade changes inventory without changing the position's endpoints or liquidity. If price crosses the upper endpoint, the position holds token1; if it later returns, that inventory becomes available to buy token0. There is no order-expiry event or reposting requirement. Fees accrue separately and do not automatically increase $L$.

For constant active liquidity and an upward move from $s_0$ to $s_1$,

$$\begin{aligned} \Delta Y &= L(s_1 - s_0), \\ \Delta X &= L(1/s_0 - 1/s_1). \end{aligned} \tag{4}$$

The taker supplies $\Delta Y$ principal and receives $\Delta X$. A downward move reverses the corresponding inventory changes. The integer implementation uses the pinned, unmodified upstream math closure documented in dependency provenance. Principal requirements round upward on entry; principal withdrawals and swap output round downward.

### 4.2 Endpoint representation

A position stores its owner, market, lower and upper ticks, liquidity, and fee checkpoints. An endpoint stores gross

attached liquidity, a signed liquidity change, and outside fee growth. Adding a position increases the lower endpoint's net liquidity and decreases the upper endpoint's net liquidity. An upward crossing applies that signed change; a downward crossing applies its negation.

The lower endpoint is inclusive and the upper endpoint exclusive. At a downward crossing of tick $t$, the stored current tick becomes $t-1$, even if the square-root price equals the endpoint exactly. This convention determines which positions are active and which fee growth belongs inside a range.

Mint, increase, burn, and collect do not iterate over the interval. Minting a range one million ticks wide still touches two endpoints. At spacing 10, both the custom interval $[-600000, 420000]$ and the full interval $[-887270, 887270]$ use this representation. Full range is not a separate liquidity product or fast path.

Width independence concerns the amount of work, not identical gas. Initializing new bitmap words, creating endpoint storage, and evaluating different fixed-point operands can change cost. The contract also imposes per-endpoint liquidity capacity and integer-domain limits. "Arbitrary width" means any valid aligned interval within those bounds.

Capital efficiency remains a separate matter. For a fixed deposit, a wider interval provides less liquidity near the current price. Cheap representation does not alter that tradeoff.

## 5. Joint execution

### 5.1 Price ordering and AMM priority

The common supply curve has a continuous component and discrete quantities at ordinary prices. For an upward execution prefix from $s_0$ to $s$, let $L(u)$ be active AMM liquidity and let $z_i$ be base executed from the ask queue at square-root price $u_i$. Before fees and integer rounding,

$$\begin{aligned} Q_X &= \int_{s_0}^{s} \frac{L(u)}{u^2}\,du + \sum_{s_0 \le u_i \le s} z_i, \\ Q_Y &= \int_{s_0}^{s} L(u)\,du + \sum_{s_0 \le u_i \le s} u_i^2 z_i. \end{aligned} \tag{5}$$

Here $Q_X$ is base delivered and $Q_Y$ is quote principal consumed. The last $z_i$ may be a partial queue fill. At a resting price, execution can increase both quantities without advancing $s$. Elsewhere, the AMM determines the marginal exchange rate. This representation combines both sources without materializing the curve as a ladder of orders.

A token1-to-token0 trade moves price upward. Its next target is the nearest of the next initialized AMM endpoint, the lowest ask, and its price limit. A token0-to-token1 trade moves downward and selects the analogous highest bid and lower endpoint.

With positive active liquidity, the curve executes from the current price toward that target. If the input budget is exhausted before the target, execution stops inside the interval. If the target is reached, the matcher first completes the curve accounting, crosses any LP endpoint there, and then consumes an ordinary queue at that price. It resumes the curve only after the queue has been exhausted.

This ordering gives the AMM all executable curve flow before the resting price. It does not let the AMM bypass an order offering a better price. Nor does it give the AMM a finite allocation at an isolated mathematical point: the integral of continuous liquidity over a zero-width interval is zero. If an order is already at the current spot price, it may execute without a preceding positive-size curve fill.

Ordinary quantity is quantized into lots. If the remaining budget cannot purchase one lot at the next resting price, execution stops. It cannot skip that order and spend the residue against a worse AMM price. This can leave input unused even when the curve beyond the order contains liquidity.

### 5.2 Execution algorithm

The following describes one trade; arithmetic and domain checks are omitted for readability.

```
budget := principal(maxInput); N := 0
while N < budget and steps < maxSteps:
    target := next endpoint, order, or limit
    if active liquidity > 0:
        trade curve toward target
        account curve principal and fees
    else:
        advance to eligible real target
    if target not reached: stop
    if target is endpoint: cross endpoint
    if target has order and budget remains:
        consume affordable FIFO prefix
        account order principal and fees
        if queue remains: stop
    if target is price limit: stop
require input > 0 and output >= minOutput
require full input if fillOrKill
commit reserve changes; settle caller
```

A step may combine a curve segment, an endpoint crossing, and execution at one ordinary price. `maxSteps` bounds matching work; it is not a count of filled makers or a gas quotation. Failure of minimum output, deadline, funding, or a strict fill requirement reverts the entire operation.

When active liquidity is zero, the matcher can cross empty price space to a real LP boundary or a level at which at least one lot is affordable. It does not move the stored price to an arbitrary taker limit after the last real level. This prevents an unused tail of a request from becoming the post-only placement anchor. A trade consuming zero input reverts. The resulting spot remains execution state, not an external price oracle.

### 5.3 Example

Consider an active interval with $L = 1000$, starting at $s = 1$, and an ask selling 25 base at $s = 1.01$, or $P = 1.0201$. A purchase continuing to $s = 1.02$ has three parts:

| Segment | Quote principal paid | Base received |
|---|---|---|
| Curve from $1$ to $1.01$ | 10 | 9.900990099 |
| Resting ask at $P = 1.0201$ | 25.5025 | 25 |
| Curve from $1.01$ to $1.02$ | 10 | 9.706853038 |
| Total | 45.5025 | 44.607843137 |

The ask increases the quantity available along this price path. Its execution leaves the AMM's principal inventory unchanged at $s = 1.01$. After the trade, the LP remains in its range, while the ask maker owns claimable quote proceeds. Reverse trading can recover base from the LP, but does not recreate the consumed ask.

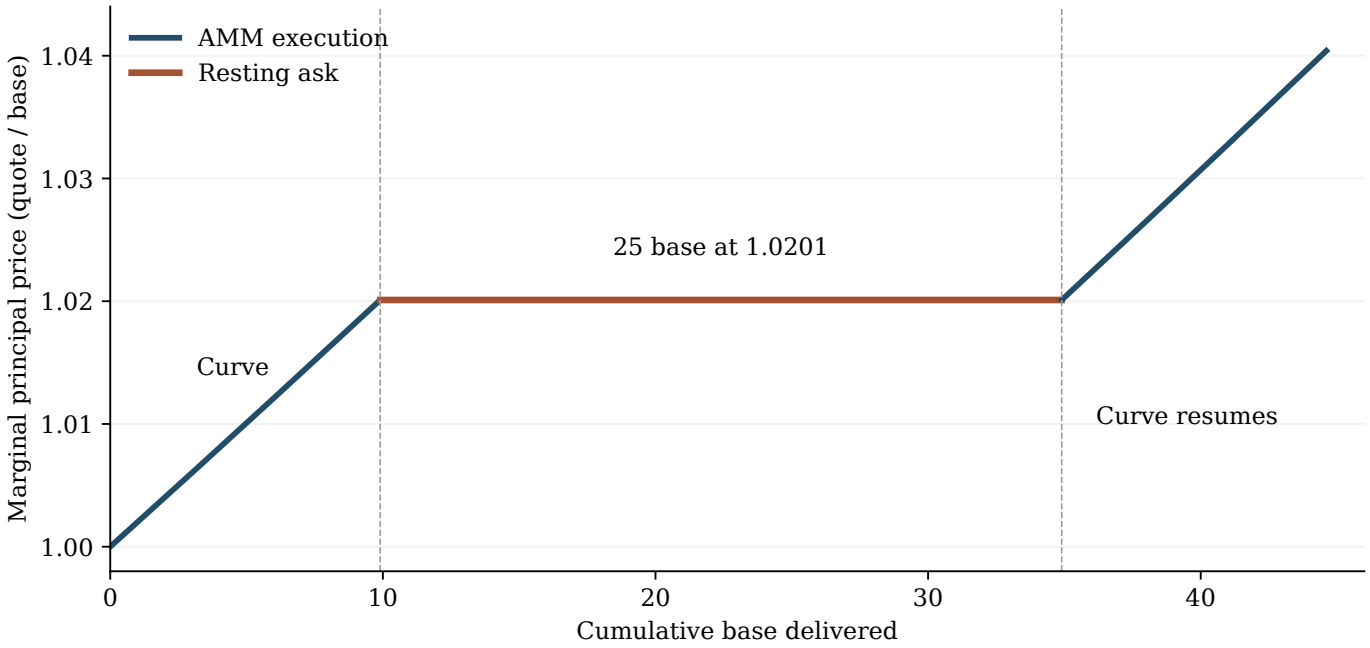



Figure 1: Marginal principal price against cumulative base delivered. The horizontal segment is ordinary liquidity. The example uses exact arithmetic and illustrative continuous prices; deployed ordinary prices must lie on the market's tick grid. Fees and token rounding are omitted from the figure.

### 5.4 Trade semantics

Trades are exact-input requests with a gross maximum input, square-root price bound, minimum output, work limit, and deadline. Ordinary execution permits a partial fill of the request, with unused input retained by or returned to the caller according to the settlement path.

The optional fill-or-kill flag requires the entire gross input budget to be spent. It is not a guarantee to acquire an exact base quantity. Integer fee conversion and order lots can make a particular gross budget unspendable in full; such a strict request reverts even if a slightly smaller partial trade is available. There is no exact-output endpoint in this release.

## 6. Uniform taker fees

### 6.1 Cumulative rounding

Let $f$ be the fee numerator and $M = 10^6$. For cumulative principal input $N$, define

$$G(N) = \left\lceil \frac{NM}{M-f} \right\rceil, \qquad N_{\max}(B) = \left\lfloor \frac{B(M-f)}{M} \right\rfloor. \tag{6}$$

A segment consuming principal $n_i$, after prior cumulative principal $N_{i-1}$, charges

$$F_i = G(N_{i-1} + n_i) - G(N_{i-1}) - n_i. \tag{7}$$

The accumulator spans AMM and ordinary execution. Upstream curve stepping is called with its internal fee parameter set to zero; the joint matcher applies Equation (7).

**Fee partition property.** For a fixed total principal $N$, the total taker fee is $G(N) - N$, independent of how that principal is divided into segments or sources.

The result follows by summing Equation (7): intermediate gross terms cancel. It applies to one trade accumulator. Separate transactions round separately, and changing a trade's execution path may change the principal and output amounts. Source allocation of the individual rounded fee atoms can also depend on segment order. The property concerns the total fee, not exact equivalence of every fee recipient under arbitrary repartitioning.

In exact arithmetic, a principal ask price $P$ corresponds to a fee-inclusive ask $P/(1 - f/M)$; a principal bid corresponds to $P(1 - f/M)$. The common fee therefore preserves price ordering across sources, subject to integer and lot granularity.

### 6.2 Allocation

For an AMM segment, let $c$ be the market's retained protocol-division remainder for the input asset. Allocation is

$$\begin{aligned} F_{\text{protocol}} &= \left\lfloor \frac{F_i + c}{d} \right\rfloor, \\ c' &= (F_i + c) \bmod d, \\ F_{\text{LP}} &= F_i - F_{\text{protocol}}. \end{aligned} \tag{8}$$

For an ordinary segment, the entire $F_i$ accrues to the protocol. That segment neither updates LP fee growth nor changes the AMM division remainder. The ordinary maker receives principal proceeds, with no additional maker fee and no maker rebate.

Starting with remainder zero, the accumulated protocol allocation on AMM fee atoms $S$ is $\lfloor S/d \rfloor$, with remainder $S \bmod d$. This follows directly by induction on Equation (8). The carry prevents repeated small AMM fills from indefinitely discarding the protocol's fractional share.

The reference schedule is fixed by market validation:

| Taker fee | AMM protocol divisor | LP share of AMM fee | Protocol share of AMM fee |
|---|---|---|---|
| 100 ppm, 1 bp | 4 | 75% | 25% |
| 500 ppm, 5 bp | 4 | 75% | 25% |
| 3,000 ppm, 30 bp | 6 | 5/6 | 1/6 |
| 10,000 ppm, 100 bp | 6 | 5/6 | 1/6 |

These are UniCLOB's pinned reference choices, following a Uniswap-style fractional protocol share. They are not a statement about the fee configuration of every Uniswap pool or version. Other permitted fee tiers require an explicit divisor between 2 and 10. An existing market's fee and divisor are immutable.

At 500 ppm and divisor 4, AMM flow pays approximately 3.75 basis points to LPs and 1.25 to the protocol; ordinary flow pays 5 basis points to the protocol. The taker pays 5 basis points in either case. LPs earn only on principal actually exchanged with the curve. An ordinary fill at a price inside an LP's interval does not itself earn fees for that LP.

### 6.3 Fee growth

For an AMM segment executed with active liquidity $L$, the input asset's global fee growth increases by

$$\Delta H = \left\lfloor \frac{F_{\mathrm{LP}} 2^{128}}{L} \right\rfloor. \tag{9}$$

This update precedes any endpoint crossing, so the interval supplying the trade receives its fees. On a crossing, each endpoint's outside growth is replaced by global growth minus its previous outside growth. Inside growth is recovered from global growth, the two endpoints, and the directional current tick. A position can therefore calculate its earnings without visiting the prices traversed since its last update.

For position liquidity $L_j$, growth difference $\Delta H_j$, and retained fractional amount $r_j$, the next whole-atom entitlement and fraction satisfy

$$\begin{aligned} W_j &= \left\lfloor \frac{L_j \Delta H_j + r_j}{2^{128}} \right\rfloor, \\ r'_j &= (L_j \Delta H_j + r_j) \bmod 2^{128}. \end{aligned} \tag{10}$$

The implementation evaluates the product with full-precision quotient and remainder operations. Retaining $r_j$ prevents repeated collection from discarding a position's sub-atom entitlement. Growth is checkpointed against the old position liquidity before its liquidity changes. Global growth addition is checked; intentional modular subtraction is confined to growth differences and outside accounting.

Global growth truncation and conservative principal rounding can leave reserves that no position can presently withdraw. These residues remain in AMM reserves. They are not reclassified as protocol revenue.

## 7. Ordinary orders and FIFO accounting

### 7.1 Collateral and lifecycle

An ask offers a whole number of base lots and locks exactly that base quantity. A bid locks quote collateral sufficient for its specified base lots, rounded upward. Orders are post-only and good until cancelled or filled. Placement cannot cross the current curve spot or the best opposing ordinary order. All collateral is held on chain; there is no margin engine or external maker credit.

At each price and side, priority follows successful chain execution order. Cancellation removes only unfilled quantity. A fill creates claimable proceeds, which the owner can realize later without the taker updating the owner's free balance during execution.

`replaceOrder` claims existing fills, cancels the remainder, and posts a new quote under the same order identifier. The new quote receives fresh time priority. Failure at any stage restores the previous state. Event consumers must distinguish successive placements under a reused identifier.

`updateOrders` accepts up to 64 records of 43 bytes each: a 32-byte order ID, signed 3-byte tick, and 8-byte target remaining quantity. It claims fills before adjusting the order. A same-price reduction or unchanged target preserves priority; a price change or quantity increase appends a fresh quote. The target is unfilled quantity at execution time, so it can replenish an order that has already filled. A zero quantity with zero tick claims and cancels. Records execute sequentially and atomically.

`cancelRemaining` cancels whatever unfilled quantity exists when the transaction executes. This avoids requiring a maker to know the exact remainder before inclusion. A fully filled or already cancelled owned order is a safe no-op; filled proceeds remain separately claimable.

### 7.2 Packed quantity tree

Each price-side queue is a four-way tree. One storage word holds four unsigned 64-bit subtree quantities. Leaves correspond to append sequence numbers. A zero subtree quantity logically clears all leaves below it, even if historical child words remain in storage.

The active root is the smallest fixed subtree covering the generation's appended sequence numbers. Up to four orders need one node. Appends 5, 17, 65, and subsequent powers-of-four transitions introduce a parent without moving existing nodes. The new parent's first child records the old root's remaining aggregate, while other child quantities start at zero. Stable node addresses and logical clearing allow the queue to grow without reviving consumed historical leaves.

A complete-level fill sets the aggregate root to zero. A partial fill removes a FIFO prefix by clearing complete subtrees and descending only through the partially consumed subtree. Individual cancellation and remaining-quantity queries follow one sequence path. Maker claims derive execution from posted quantity, cancelled quantity, and the remaining leaf entitlement.

For a generation containing $n$ appended sequence positions, path depth is at most $\lceil \log_4 n \rceil$, with a one-node minimum. The implementation caps depth at 16 and permits $2^{32}$ appends per generation. Aggregate remaining quantity cannot exceed $2^{64} - 1$ lots. Queue history does not shrink on fills, so a sparsely populated but long-lived queue can retain a deep path.

Only an empty queue can begin a new generation. Replacement into an empty queue can restart early; ordinary appends roll over at sequence capacity when empty. An old-generation

order has zero remaining quantity, but its filled proceeds remain claimable through its order record. These rules separate execution history from funds ownership.

The savings concern taker work at a level. Posting, cancelling, and claiming still require maker transactions. Aggregate execution moves part of the settlement cost to the owner claiming the proceeds; it does not remove that lifecycle cost.

### 7.3 Integer price and proceeds

The square of a small square-root price is represented in Q192. At larger square-root prices, the quote rate uses a conservatively truncated Q128 representation. Forward quotes and affordability calculations use the same rate, avoiding an inconsistent reciprocal. Placement requires each lot to quote to at least one quote atom, and bounds collateral amounts to the supported integer domain.

A buyer consuming asks pays quote rounded upward; a seller consuming bids receives quote rounded downward. Ask claim proceeds use differences of cumulative executed-quantity quote floors. Bid cancellation refunds use differences of cumulative cancelled-quantity quote floors. Thus repeated claims or cancellations do not repeatedly obtain a fresh favorable rounding of the same entitlement. Bid makers receive exact executed base lots.

The difference between aggregate taker payments and individually rounded maker entitlements remains in order reserves. A fully cancelled bid can leave a quote atom from its initial upward-rounded collateral. The current design does not sweep these residues to the protocol. Rounding is conservative and economically visible at small lot sizes; market configuration should not treat atom-level dust as negligible by definition.

## 8. Custody and contract architecture

### 8.1 Singleton accounting

Markets share a singleton custody contract, while retaining separate reserve ledgers. For each token, let $B$ be total free balances, $A$ AMM reserves across markets, $O$ order reserves, and $R$ allocated protocol fees. The accounting identity is

$$\begin{aligned} \mathcal{L} &= B + A + O + R, \\ \text{balanceOf (singleton)} &\geq \mathcal{L}. \end{aligned} \tag{11}$$

Here $A$ includes LP principal, unclaimed LP fees, and rounding residues; $O$ includes unfilled collateral, executed but unclaimed proceeds, and residues. Unsolicited token donations increase physical custody without increasing anyone's entitlement. Tests without donations require equality.

Matching transfers value between these categories. An AMM fill adds principal and LP fees to one AMM reserve, subtracts output from the other, and credits the protocol allocation. An ordinary fill performs the corresponding principal transfer within order reserves and credits the entire fee to the protocol. Claiming or cancelling transfers entitlement from a reserve to a user balance or wallet credit. None of these operations creates withdrawable value from a donation.

The identity is a conservation condition, not a complete correctness proof. An implementation could conserve total value while assigning it to the wrong owner. FIFO entitlement, LP fee attribution, authorization, and conservative rounding require separate properties.

### 8.2 Funded and wallet settlement

The funded interface operates on internal free balances. Makers can deposit once, replace or cancel orders, claim proceeds, and reuse funds without an ERC-20 transfer for each action. Withdrawal transfers the chosen free balance back to a wallet. Resting orders remain collateralized regardless of how their owner funded them.

A taker need not deposit. `swapWallet` executes the same matcher, pulls only actual consumed input from the caller, and pays output directly to the recipient. `executeWallet` extends wallet settlement to an atomic batch of up to 64 allowlisted LP, order, and trade operations. It accumulates signed deltas per token and settles each token once at the end. Existing internal balances remain untouched; insufficient wallet allowance cannot silently consume a funded balance instead.

A positive delta is a debt to the exchange; a negative delta is output owed to the recipient. Token-specific liability changes are applied before external transfers, under the exchange guard. Intermediate deltas must fit the signed arithmetic domain. The batch owner, touched-token list, and deltas are cleared before returning, allowing subsequent independent calls in the same transaction.

Transient storage supplies transaction-scoped state for the batch and reentrancy guard. Its rollback semantics follow call-frame reverts, but a successful call does not automatically clear it before a later call in the same transaction; explicit cleanup is therefore part of settlement correctness [5].

There are no arbitrary callbacks or nested wallet batches. Matching itself makes no external token calls. Inbound custody deltas and outbound custody and recipient deltas must match the transfer amount. The supported asset model is an exact-transfer, non-rebasing ERC-20; balance checks do not make arbitrary or dishonest token code safe.

### 8.3 Singleton versus separate markets

The singleton allows one funded balance per asset, shared wallet settlement, and reserve reclassification without inter-market token transfers. A separate market contract would isolate custody at the contract boundary and make individual market inspection simpler, but would require additional deployment and transfer or settlement machinery for cross-market operations.

UniCLOB selects the singleton because repeated maker activity and multi-operation LP workflows benefit from shared custody. This choice also concentrates implementation risk: a defect in shared authorization or reserve accounting can affect multiple markets. Internal ledger separation is not the same fault boundary as separate custody contracts. No benchmark of an otherwise identical separate-contract implementation has been performed.

The core links three immutable libraries: `ConcentratedLiquidity`, `OrderBookAccounting`, and `WalletSettlement`. Their public library functions execute in the singleton's storage context. They are implementation modules, not independently governed vaults. Deployment

verification must cover the core, linked addresses, and library bytecode together.

The measured core runtime is 24,532 bytes, leaving 44 bytes below the 24,576-byte limit specified by [6]. This is a material constraint on further core features. The router, position manager, quoter, and lens remain separate contracts. The core has no upgrade mechanism; the immutable fee collector can withdraw allocated protocol fees but cannot change market rules or take user reserves.

## 9. Sparse price discovery and work bounds

For an aligned tick $t$, the price index uses the compressed key

$$k = t/h - \operatorname{trunc}\,(t_{\min}/h). \tag{12}$$

The truncation is toward zero, matching Solidity signed division. Search from an unaligned tick rounds upward for an ascending query and downward for a descending query, including at negative ticks.

The index stores occupancy bits in leaves and summaries of nonempty children above them. Spacings of at least 28 fit within a two-level structure; finer grids use three levels. A lookup finds the next populated point without scanning every empty word. LP endpoints, asks, and bids have distinct indexes.

Let $K$ denote visited matching events and $n_i$ the append history at an ordinary level. Price-index lookup has a fixed domain bound. Curve and boundary work grows with $K$; partial order-prefix work additionally depends on bounded tree paths. A useful schematic bound is

$$T_{\text{match}} = \left(\mathcal{O}K + \sum_{i \in \text{ partial levels}} \log_4 n_i\right), \tag{13}$$

with tree depth at most 16. Complete-level consumption does not add one operation per maker. Position modification has work independent of range width.

A large price move through one active interval can therefore be cheap, while a small move through many distinct endpoints can be expensive. The index skips empty space, not accounting obligations. Dense endpoints require active-liquidity and fee-growth updates; distinct order prices require level execution. Permissionless participants can increase that fragmentation. The work limit bounds a request's exposure, at the cost of a partial fill or revert when the requested outcome cannot be reached within it.

## 10. Integration contracts

The core exposes account-owned positions and direct wallet execution. The periphery adds common wallet and application conventions without changing the matching rules.

| Contract | Responsibility |
|---|---|
| Swap router | Exact-input single-market and multi-hop swaps, native wrapping, permit support, refunds, and payer-attributed events |
| Position manager | ERC-721 position ownership, token-budget liquidity calculation, principal withdrawal, and fee payout |
| Quoter | Simulation through the actual matcher with reverted state changes |
| Market lens | Bounded ordinary depth, order views, and liquidity-budget previews |

The router supports one to eight connected, distinct markets. A multi-hop route requires full-budget execution at each hop and enforces a final minimum output atomically. Repeated market IDs are rejected, keeping the route consistent with independent rollback quotes of each market. Single-market partial fills return call-local unused input. Pre-existing router donations are not included in refunds. Native currency is wrapped at the periphery; the core handles ERC-20 assets only.

The position manager owns core positions and assigns the corresponding IDs to NFTs. Desired token budgets are converted into an affordable liquidity amount with full-precision arithmetic; ERC-20 funding pulls required principal rather than the entire desired budget. Increasing a position pays previously accrued fees to its NFT owner, even when an approved operator supplies the new capital. Decreasing liquidity pays principal and accrued fees immediately; principal minimums and reported principal amounts exclude the fee component. Empty NFTs can be burned after whole-atom fees are settled.

These are transferable managed positions, not wrappers that can import any pre-existing core position. Direct core positions remain a separate ownership path. There is no NFT permit, generic manager batch, or exact-output router interface in the current release.

The quoter invokes `quoteTrade`, which executes the production matcher and deliberately reverts with a typed result. The quoter accepts only the expected selector and payload length; other errors propagate. State changes are rolled back and token funding is unnecessary. Because simulation writes state before reverting, this is an `eth_call` interface, not an EVM `STATICCALL`-compatible view. A quote does not validate the caller's eventual allowance or recipient behavior. Integrators should simulate the actual funded transaction as well.

The lens returns ordinary order depth only. AMM depth must be reconstructed from price, active liquidity, and endpoint changes, or evaluated through quotes. Rendering the ordinary levels alone as total market depth would omit the underlying curve. Pagination should use a fixed block tag, since moving market state can otherwise duplicate or omit levels between calls.

Interfaces, exported ABIs, event semantics, packed order-update encoding, and deployment scripts are described in the Solidity interfaces. Events support off-chain discovery and indexing; they do not substitute for canonical contract state when determining execution.

# 11. Experimental evaluation

## 11.1 Method

The principal comparison uses local deployments of unmodified published artifacts from `@uniswap/v3-core@1.0.1` and `@uniswap/v4-core@1.0.2`. V3 pools are created by their factory; v4 uses its real unlock and settlement path. Minimal benchmark adapters supply callbacks and token settlement. They are not the public Uniswap router or NFT position manager.

UniCLOB and the adapters compile with Solidity 0.8.30, Cancun, via IR, and 200 optimizer runs. Published v3 artifacts retain compiler 0.7.6. Published v4 artifacts retain compiler 0.8.26, Cancun, via IR, and 44,444,444 optimizer runs. The comparison therefore measures published implementations, not a same-compiler algorithm experiment. Artifact hashes and licenses are preserved in benchmark provenance.

Fixtures start at raw price 1 with 18-decimal assets, spacing 10, and $L = 10^{24}$ per seeded position. Approvals and nonzero user balances are established. Each measured call begins from committed storage with cooled contract, library, and token access. Different venues do not first mutate the same token balance slots inside one measured test. The clock advances after setup so v3 must process a later-block swap observation.

All comparison paths use an effective 500 ppm taker fee. UniCLOB and v3 use divisor 4. V4 uses a 125 ppm directional protocol fee and a 375 ppm LP fee, composing to 500 ppm under its integer formula; hooks are disabled. Input spending is checked exactly. Compared outputs allow up to 1,000 atoms of difference, or $10^{-15}$ tokens in these fixtures, because rounding conventions differ.

Reported numbers are **callee execution gas before refunds**. They exclude transaction intrinsic and calldata charges, initial user approvals, and L2 data fees. Wallet rows include the settlement transfers made by the measured call. Funded rows leave output internal; v4 claims rows use existing ERC-6909 claims. The adapters and UniCLOB core also differ in ownership and limit checks. These are controlled workflow comparisons, not estimates of website transaction receipts.

## 11.2 LP and taker costs

| Operation | UniCLOB funded | V4 claims | UniCLOB wallet | V3 wallet | V4 wallet |
|---|---|---|---|---|---|
| Mint narrow, $[-60, 60]$ | 297,775 | 197,710 | 423,781 | 305,286 | 255,577 |
| Mint custom wide, $[-600000, 420000]$ | 342,486 | 197,940 | 468,492 | 305,384 | 255,807 |
| Mint full range | 342,716 | 198,170 | 468,722 | 305,482 | 256,037 |
| Increase full range, no prior fees | 78,852 | 84,178 | 136,447 | 112,832 | 107,845 |
| Collect full-range fees | 83,596 | 67,385 | 117,577 | 101,563 | 75,051 |
| Burn full position and collect | 159,761 | 111,348 | 219,493 | 172,800 | 126,680 |
| Close full range, reopen $[-600, 600]$ | 305,272 | 231,098 | 364,989 | 362,296 | 246,430 |
| First upward AMM swap | 113,560 | 98,960 | 152,271 | 108,976 | 117,456 |
| Established upward AMM swap | 79,360 | 64,760 | 118,071 | 74,776 | 83,256 |
| Large move through full range | 79,945 | 174,907 | 118,656 | 221,030 | 193,403 |
| Cross 17 interior endpoints, one fee history | 663,789 | 596,767 | 702,500 | 744,668 | 615,263 |
| Same traversal, both fee histories | 1,040,142 | 935,067 | 1,078,853 | 1,083,037 | 953,563 |

The established swap consumes $10^{18}$ gross input. The large move consumes $10^{25}$ and traverses empty tick space inside a full-range position. The endpoint case consumes $2 \times 10^{23}$ against 16 added adjacent ranges with 17 distinct interior endpoints. Each row is a specific state history, not a universal bound. Complete fixtures and additional cases are in the benchmark suite.

The ordinary single-interval wallet swap is more expensive in UniCLOB than either measured Uniswap path. New LP positions also cost more. The additional order indexes, reserve categories, position ownership, and settlement machinery have a cost even when a particular trade uses only the AMM.

The large-move result is different: summary indexes avoid repeated empty-word traversal. This advantage disappears as actual initialized boundaries replace empty space. The both-fee-history traversal illustrates the remaining bottleneck, exceeding one million gas in UniCLOB because both outside-growth fields must be maintained. The results support a conditional advantage for sparse traversal, not general gas dominance.

## 11.3 Maker costs

Maker benchmarks use 500 ppm fees and spacing 1. Values below use funded balances unless stated otherwise. Uniswap's core AMM position operations do not provide an equivalent native FIFO order lifecycle, so no artificial one-to-one comparator is assigned.

| Operation | Execution gas |
|---|---|
| First ask at a new level | 231,193 |
| First bid at a new level | 233,254 |
| Post a bid and ask | 452,365 |
| Post ten orders | 1,457,138 |
| Cancel an ask | 53,765 |
| Cancel a bid | 55,854 |
| Replace two orders, existing prices | 164,973 |
| Replace two orders, new prices | 244,421 |
| Replace ten orders, existing prices | 605,499 |
| Replace ten orders, new prices | 980,275 |
| Packed ten-order update to new prices | 962,865 |
| Packed ten-order size reduction | 302,826 |
| Packed ten-order unchanged target | 142,909 |
| Packed ten-order claim and cancel | 343,721 |
| Packed 64-order update to new prices | 5,942,040 |
| Wallet-settled packed ten-order update to new prices | 1,027,828 |
| Claim one filled bid | 32,452 |
| Claim ten filled asks | 235,281 |

Repricing is substantially more expensive than reducing or cancelling existing quotes. Packed calldata removes ABI overhead, but cannot eliminate storage initialization at new levels. Even an unchanged target requires authorization, state inspection, and claim handling. A market maker should use local state to avoid submitting unnecessary no-op updates.

Separate queue-capacity tests measure a full 4,096-maker level fill at 112,472 gas, partial-prefix execution at 142,598 gas, and the 4,097th append at 272,588 gas. These are structural queue tests with their own fixture, not entries in the spacing-1 maker comparison. They demonstrate that maker count at one level does not become a taker loop, while expansion and individual maintenance remain paid work.

### 11.4 Integration overhead

| Periphery operation | Execution gas |
|---|---|
| Single-hop router swap, established core approval | 163,596 |
| Single-hop router swap, first core approval | 186,258 |
| Two-hop router swap | 371,968 |
| NFT mint at new endpoints | 430,051 |
| NFT full-range mint at existing endpoints | 347,091 |
| NFT increase with historical fee payout | 287,368 |
| NFT full removal with fee payout | 295,066 |
| NFT fee collection | 180,289 |
| Matcher-backed single-hop quote | 72,074 |

Within matched periphery fixtures, routing adds 49,047 gas over a direct core wallet swap. Token-budget NFT minting adds 117,997 gas over a direct wallet mint receiving a precomputed $L$. The latter also adds transferable ownership and budget calculation, so these are different product functions. The lower full-range NFT mint number reflects existing endpoints, not a general advantage for wide ranges. Quote calls normally run through RPC simulation and do not require a paid transaction. See the periphery fixtures.

### 11.5 On-chain feasibility

The measured operations fit an EVM execution model, but gas alone does not establish a high-frequency venue. A 5.94-million-gas batch of 64 reprices is substantial. Repeated new-price updates can consume more block capacity than taker execution. An L2 can reduce the monetary price of that work without removing sequencing latency, cancellation races, data publication costs, or shared block limits.

For an execution gas amount $g$ and execution gas price $p$ in gwei, the execution component is $gp\,10^{-9}$ native tokens. A complete cost estimate must additionally use the actual transaction's intrinsic/calldata treatment, refunds, and chain-specific data or operator charges. These tables omit those components. The repository's earlier dollar-cost study is pinned to an older revision and is not a current price quotation for this implementation.

The present evidence is strongest for persistent LP positions, ordinary taker transactions, and bounded maker updates on a sufficiently inexpensive EVM chain. It does not establish centralized-exchange latency or sustained HFT throughput. Those require workload replay and deployed-chain measurement under contention.

## 12. Security properties and validation

The central accounting properties are reserve conservation, owner authorization, correct FIFO execution, fee attribution to the supplying liquidity, and atomic settlement. The test suite exercises them through scenario tests, randomized inputs, stateful handlers, independent curve comparisons, and mutation tests.

The release suite passes 359 Foundry tests covering core, periphery, workflows and reference comparisons. Twenty-six stateless fuzz tests run 4,096 cases each. Four stateful invariants run 256 campaigns of 256 calls, totaling 262,144 handler calls with no unexpected handler reverts. The recorded run also checked 191 gas snapshot entries; these can be regenerated locally from the benchmark tests. Superseded prototype tests are excluded from these counts.

| Coverage scope | Lines | Branches | Functions |
|---|---|---|---|
| Current core and seven libraries | 752/775, 97.03% | 189/210, 90.00% | 91/91, 100% |
| Six periphery source files | 281/286, 98.25% | 62/65, 95.38% | 41/41, 100% |

Coverage instrumentation uses a separate build, and Foundry's minimum-IR mode can misattribute assembly source locations. Raw counts are retained rather than adjusted to infer coverage. Assertions exercise several such assembly paths through their outputs and errors. Coverage is evidence of execution, not proof of correct behavior.

The validation includes 21 killed core mutants and 12 killed periphery mutants. Static analysis reports no high or medium Slither findings and no high Aderyn findings, with lower-severity findings retained for review. Runtime size and snapshot checks use optimized artifacts separately from coverage. These checks passed in the recorded CI run.

Several assumptions remain fundamental. A token can pause, blacklist, rebase, or lie about balances outside the exchange's control. Reentrancy guards and exact-transfer checks constrain interactions but do not repair token semantics. Shared custody increases the consequences of a common accounting defect. Immutable contracts remove an upgrade administrator but also remove an in-place patch path. Independent review, deployment verification, and adversarial operational testing remain necessary before capital is exposed.

## 13. Market structure and limitations

### 13.1 LP economics

An LP receives the same principal inventory rule as a concentrated-liquidity position, but its flow distribution differs. Ordinary depth absorbs volume at fixed prices and can reduce the movement required for a given taker quantity. That can improve the taker's realized execution along an otherwise unchanged path. It also diverts some volume away from LP fee generation. Whether LP returns improve depends on the composition of flow, fees, inventory risk, and maker behavior.

AMM priority does not eliminate adverse selection. A public curve can remain stale relative to external markets, exposing it to informed arbitrage. Loss-versus-rebalancing is one framework for measuring that cost; adding a FIFO queue does not by itself remove its cause [4].

There is no external oracle, adaptive fee controller, or minimum LP residence time. The design does not claim resistance to sandwiching or just-in-time liquidity. Slippage bounds and deadlines constrain a user's accepted outcome but do not impose fair transaction ordering.

### 13.2 Maker incentives and constraints

Ordinary makers control price and size and retain price-time priority among ordinary orders. They receive no share of the taker fee. Their return must therefore come from their execution price and inventory strategy after maintenance costs. Requiring curve execution before a resting price limits how much flow a maker obtains merely by joining a price that has not yet been reached. Price improvement within the post-only constraints can bring the maker forward on the execution path.

The minimum tick spacing gives roughly one basis point between adjacent principal prices. Markets needing sub-basis-point maker quotes cannot express them with the current grid. Lot size introduces a second granularity constraint. Neither parameter can be changed in an existing market; a different configuration creates a separate market and may fragment liquidity.

FIFO means chain execution order. It does not mean RPC arrival order, equal network access, or protection against sequencer preference. Makers face the usual race between a cancellation and an incoming trade. There is no self-trade prevention, delegated session-key system, native order expiry, iceberg order, stop order, or OCO instruction. Fully collateralized post-only orders and bounded atomic updates are the implemented scope.

### 13.3 Liquidity quality and protocol incentives

The protocol receives a larger fraction of fees from ordinary execution than from AMM execution. The matcher does not choose sources to maximize protocol revenue: it follows the fixed price path. Nevertheless, the revenue policy creates an economic distinction that should be evaluated rather than hidden. It may affect the relative attractiveness of passive and active provision, market configuration choices, and third-party incentives.

Persistent AMM liquidity provides executable depth only while it exists in an active range. LPs can withdraw, and orders can cancel. The protocol does not guarantee continuous depth, an external fair price, or a particular spread. A

singleton and a common interface make liquidity accessible; they do not supply liquidity independently of participants.

## 14. Conclusion

UniCLOB treats automated liquidity and a central limit order book as components of one on-chain market. A range position remains a reusable inventory rule; a limit order remains a finite, cancellable quote with FIFO priority. Continuous execution reaches an ordinary price before consuming its queue, and a single cumulative fee applies across both sources. Separate reserve and fee accounting preserve the ownership consequences of each type of fill.

The implementation removes range width and the number of makers at a fully consumed level as linear execution costs. Its remaining costs are concentrated in initialized boundaries, distinct order prices, maker storage updates, and wallet integration. The measurements show where this representation is useful and where a dedicated AMM remains cheaper. The next empirical questions concern sustained maker workloads, liquidity composition, and realized LP and maker returns under live ordering conditions.

## References


1. Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. *Uniswap v3 Core*. March 2021.
2. Hayden Adams et al. *Uniswap v4 Core*. August 2024.
3. Clober. *Introduction: LOBSTER*.
4. Jason Milionis, Ciamac C. Moallemi, Tim Roughgarden, and Anthony Lee Zhang. *Automated Market Making and Loss-Versus-Rebalancing*. arXiv:2208.06046, originally 2022.
5. Alexey Akhunov and Moody Salem. *EIP-1153: Transient storage opcodes*.
6. Vitalik Buterin. *EIP-170: Contract code size limit*.
7. Alex Beregszaszi et al. *EIP-5656: MCOPY — Memory copying instruction*.
8. UniCLOB. *Implementation and test fixtures*. Revision 27e03b0, 20 October 2026.


## Appendix A. Reproduction

The contract identifiers below map the mechanism to the measured implementation.

| Mechanism | Implementation or evidence |
|---|---|
| Matching, cumulative fee allocation, reserve updates | ConcentratedRangeBook.sol |
| Position inventory, endpoint crossing, LP fee growth | ConcentratedLiquidity.sol |
| Market validation and ordinary-order accounting | OrderBookAccounting.sol |
| FIFO prefix consumption and generations | AdaptiveFifoQueue.sol |
| Spacing-compressed price lookup | TickTree.sol |
| Gross/principal fee conversion | FeeMath.sol |
| Transient batch settlement | WalletSettlement.sol |
| Exact-input router | RangeBookRouter.sol |
| NFT positions and budget inversion | RangeBookPositionManager.sol, LiquidityAmounts.sol |
| Matcher simulation and bounded views | RangeBookQuoter.sol, RangeBookLens.sol |
| Published-venue comparison fixtures | VenueComparisonGas.t.sol |
| Maker workflows and capacity costs | MakerGas.t.sol, QueueCapacityGas.t.sol |

The pinned build and test dependencies are listed in the README. From the repository root:

```
forge fmt --check
forge build --sizes
FOUNDRY_PROFILE=ci forge test
forge test --match-contract GasTest \
  --gas-snapshot-emit true
```

These commands reproduce engineering checks, not a live-chain throughput experiment. Benchmark settings, source hashes, and runtime templates should be retained when publishing revised measurements. The Cancun target is required for transient storage and the quoter's memory-copy instruction [7].